

Key Points

- **Recommended Architecture:** The Multivariate LSTM Fully Convolutional Network (MLSTM-FCN) is a robust hybrid model that combines LSTM for capturing long-term dependencies in sequences with a Fully Convolutional Network (FCN) for extracting local features. It has been proven effective for multivariate time series classification tasks, which align closely with your problem's structure of variable-length 1D sequences of high-dimensional vectors (e.g., 150 features per instruction with repeating patterns). This model can be adapted as the feature extractor in your RL actor-critic network, replacing the classification head with outputs for policy π (softmax over actions) and value V (linear regression). Research suggests it outperforms pure LSTM in similar settings due to the complementary CNN component, but you can ablate the FCN if preferring pure LSTM.
- **Why This One?:** It handles multivariate sequential data with repeating patterns (similar to NLP or time series like sensor readings), is practical and proven on 35+ datasets, and prioritizes learning quality with good balance between speed and precision. Bidirectional LSTM is not used here (unidirectional for efficiency), but you can experiment with it. No overfitting concerns as per your note on random circuit generation.
- **Adaptation to RL:** Use the model to process the padded sequence input, take the concatenated embedding from LSTM and FCN, then add FC layers for π and V . For variable lengths, pad to a max circuit length (e.g., 100-500 instructions) with zeros and mask in LSTM if needed for accuracy.
- **Implementation Availability:** Open-source Keras code is available on GitHub, easily portable to C++ with libtorch (PyTorch equivalent). The original paper provides conceptual details for reproduction.

Architecture Overview

The model processes input sequences (batch, time_steps, features) through parallel branches:

- **LSTM Branch:** Dimension shuffle to treat as (batch, features, time_steps) for

multivariate handling, then LSTM layer (8-128 cells, default 128), followed by dropout (0.8).

- **FCN Branch:** 3 1D convolutional blocks with filters (128, 256, 128), kernel sizes (8, 5, 3), batch norm, ReLU, and optional squeeze-excitation for feature weighting.
- **Combination:** Global average pooling on each branch, concatenate outputs (256-dim embedding), then dense layer for final output.
- Input Size: Features = 150 (your IRS), time_steps = padded variable length.
- Total Params: ~100k-500k depending on LSTM cells, suitable for your scale without size constraints.

Hyperparameters

- LSTM Cells: Grid search 8, 64, 128 (start with 128 for balance).
- Dropout: 0.8 on LSTM.
- Conv Filters/Kernels: Fixed as above for proven performance.
- Learning: Adam optimizer, categorical crossentropy (adapt to actor-critic losses like advantage for A2C/PPO).
- Batch Size: 128, Epochs: 250 (adjust for RL rollouts).
- No attention in base model (simple to add if desired, but avoided per your preference).

Performance Notes

- Achieves state-of-the-art on UEA multivariate time series archive (e.g., 5-10% accuracy gain over baselines like pure LSTM or CNN).
 - Strong on variable-length data with padding; handles repeating patterns via LSTM memory and CNN local filters.
 - In RL adaptation, it should learn well on random sequences, comparable to your TCN but potentially better on long dependencies.
-

The Multivariate LSTM Fully Convolutional Network (MLSTM-FCN) is an extension of the

LSTM-FCN model, which is a deep learning architecture for time series analysis.

LSTM-FCN model, specifically designed for multivariate time series data. It augments the FCN block with a squeeze-and-excitation mechanism to improve feature selection, while the LSTM branch captures temporal dependencies. The model is unidirectional LSTM to maintain efficiency, but bidirectional can be implemented by duplicating the layer and concatenating forward/backward hidden states (increasing params $\sim 2x$). For your RL setup, this serves as a drop-in replacement for the feature extractor in actor-critic networks—process the encoded circuit sequence to a fixed embedding, then use separate heads for policy (softmax) and value (linear). If pure LSTM is preferred, disable the FCN branch and rely solely on LSTM output after global pooling.

Detailed Model Structure

The input is a padded multivariate sequence (`batch_size`, `time_steps`, `num_features=150`). Padding to a fixed max length (e.g., based on typical circuit sizes) ensures compatibility; use zero-padding and optional masking in LSTM to ignore pads.

- **Input Preprocessing:**

- Pad sequences to `max_length` (variable handled via dynamic batching or fixed pad).
- No normalization mentioned in paper, but recommend scaling parameters (your 3 optional params) to $[-1,1]$ or $[0,1]$ for stability, keeping one-hot gates and qubit involvement as is ($\pm 1, 0$).
- Dimension shuffle for LSTM: Transpose to (`batch_size`, `num_features`, `time_steps`) to treat features as 'time' for multivariate processing, enabling the LSTM to learn cross-feature dependencies.

- **LSTM Branch:**

- LSTM layer with hidden size/cells tuned via grid (8, 64, or 128; paper recommends 128 for most datasets).
 - 1 layer (no stacking for simplicity, but you can add 1-2 more for depth if needed).
 - Dropout = 0.8 applied to LSTM output to prevent overfitting (though not a concern for you).
 - Output: Last hidden state or mean pool over sequence (dim ~128).
- **FCN Branch:**
 - 3 temporal convolution blocks:
 - Conv1D: 128 filters, kernel_size=8, stride=1, padding='same'.
 - BatchNorm (momentum=0.1, epsilon=0.001).
 - ReLU activation.
 - Conv1D: 256 filters, kernel_size=5.
 - BatchNorm, ReLU.
 - Conv1D: 128 filters, kernel_size=3.
 - BatchNorm, ReLU.
 - Squeeze-and-Excitation (SE) block: Global average pool, FC to reduce dim (ratio=16), sigmoid, multiply back for feature gating.
 - Global average pooling to fixed vector (dim 128).
- **Fusion and Output:**
 - Concatenate LSTM and FCN outputs (~256 dim total).
 - Dense layer to num_classes (for classification; adapt to action_space for π and 1 for V).
 - Total model size: Compact (~200k params with 128 cells), fast inference.

This hybrid setup is ideal for your repeating pattern data, as CNN extracts local gate patterns (e.g., subit interactions) while LSTM models global sequence context (e.g., circuit

patterns (e.g., qubit interactions), while LSTM models global sequence context (e.g., circuit flow). For bidirectional, run LSTM forward and backward, concat hiddens (doubles dim, but enhances context).

Implementation and Adaptation Notes

- **Code Base:** Keras/TensorFlow (GitHub repo linked below). Key files: model generators (`generate_model()`, `generate_model_2()` for MALSTM-FCN with attention if simple). Train with `batch_size=128`, `epochs=250`. Port to libtorch by rewriting `nn.LSTM` and `nn.Conv1d` equivalents.
- **RL Adaptation:** In A2C/PPO, use as shared backbone for actor/critic. Input: Padded circuit tensor. Output: From concat embedding, add actor head (FC $\rightarrow$ softmax for π) and critic head (FC $\rightarrow$ linear for V). Train with RL losses (policy gradient + value MSE). If pure LSTM needed, remove FCN/SE, use LSTM output directly (last hidden or pooled).
- **Handling Variable Length:** Paper assumes fixed-length datasets, but model supports variable via padding (common in time series). In code, use `tf.keras.preprocessing.sequence.pad_sequences` or libtorch equivalent. For efficiency, sort batches by length and use packed sequences in LSTM.
- **Hyperparameter Tuning:** Fixed conv params as per paper; tune LSTM cells (start 128), dropout (0.5-0.8), learning rate (0.001 Adam). No attention in base (avoids transformers), but MALSTM variant adds simple attention if desired (easy, no full transformer).
- **Performance Insights:** Outperforms baselines (e.g., pure LSTM, ResNet) on 35 UEA datasets by 2-15% accuracy, with fast training (hours on GPU). In your setup, expect similar gains over pure LSTM on complex patterns; compare vs. TCN as planned. Evidence leans toward hybrid for better feature learning on multivariate data like yours.

Key Citations:

- Multivariate LSTM-FCNs for time series classification [sciencedirect.com](https://www.sciencedirect.com)

- LSTM Fully Convolutional Networks for Time Series Classification [arxiv.org](#)
- GitHub Implementation: <https://github.com/titu1994/MLSTM-FCN> [github.com](#)
- Implementing LSTM-FCN in PyTorch [dzlab.github.io](#)

↳ Bidirectional LSTM variants

↳ TCN vs MLSTM-FCN comparison

↳ More concise implementation steps